أكاديمية طويق
Tuwaiq Academy

Data Representation in AI (NumPy & Matrices)

Unit 2 project

Prepared by:

Fahad Alghofaili

Ahmed Alshimshir

Mansour Alhenaki

Mohammed Alburaq

I. Abstract

Artificial Intelligence (AI) systems fundamentally rely on mathematical structures to represent and process data. Before applying any Machine Learning (ML) algorithm, raw data must be transformed into structured numerical representations. In ML, datasets are typically represented as matrices where rows correspond to samples and columns correspond to features. This research investigates the mathematical foundations of data representation in AI using NumPy and matrix operations. A real-world educational dataset, **xAPI-Edu-Data**, obtained from Kaggle, is used to demonstrate how raw tabular data is transformed into numerical matrix form and analyzed before being used in a Machine Learning model. The study highlights the importance of matrix shape, dimensional consistency, and numerical encoding in building reliable AI systems.

Table of Contents

I.Abstract 2
Chapter 1: Introduction 4
Chapter 2: Background 4
Chapter 3: Literature Review 4
Chapter 4: Methodology 5
4.1 Proposed Process 5
4.2 Dataset 5
Chapter 5: Experimental Design and Settings 6
5.1 Implementation Details 6
Chapter 6: Results and Discussion 6
6.1 Results 6
6.2 Discussion 7
Chapter 7: Conclusion 7
References 8

Chapter 1: Introduction

Artificial Intelligence systems rely entirely on mathematical representations of data. Before any learning algorithm can operate, datasets must be structured into numerical formats. In Machine Learning, datasets are typically stored as matrices where rows represent samples and columns represent features. Modern AI frameworks such as NumPy, pandas, and scikit-learn provide tools for transforming raw datasets into structured matrix representations. In this research, we use a real-world educational dataset (xAPI-Edu-Data) from Kaggle to demonstrate how datasets are converted into NumPy matrices and analyzed mathematically before being used in Machine Learning models.

Chapter 2: Background

This chapter introduces the mathematical and computational foundations required to understand data representation in Artificial Intelligence

1. **Mathematical Definitions**
   - Scalar: A single numerical value.
   - Vector: A one-dimensional array of numbers.
   - Matrix: A two-dimensional array with rows and columns.
   - Tensor: A multi-dimensional generalization of matrices.

2. **Shape and Dimensions**
   - For a dataset matrix $X \in R^{(m \times n)}$:
     m = number of samples (rows)
     n = number of features (columns)

3. **Libraries.**
   - **NumPy**: Numerical matrix operations
   - **pandas**: Dataset loading
   - **scikit-learn**: ML model training

Chapter 3: Literature Review

This chapter covers three papers that relate to our project.

The first one is Micha, E. [1] presented an article explaining how proportional representation can be applied in AI beyond traditional voting systems. The paper focused on two main domains: clustering and AI alignment. In clustering, it ensures that large and cohesive groups are fairly represented in cluster assignments. In AI alignment, particularly in RLHF, it proposes

using committees of reward functions to proportionally reflect diverse human preferences. Overall, the work argues that proportional representation offers a rigorous framework for aligning AI systems with diverse viewpoints. Additionally, Harris et al. [2] described array programming as a powerful and expressive approach for manipulating vectors, matrices, and higher-dimensional data. They highlighted NumPy as the core array programming library in Python and a fundamental component of scientific research across many disciplines. The paper emphasized NumPy's role in major scientific achievements and explained how its core array concepts enable efficient data analysis. Additionally, NumPy serves as the foundation of the scientific Python ecosystem and acts as an interoperability layer supporting modern scientific and industrial applications. Finally, Bowen, C. [3] presented a study showing how matrix methods can be applied to atomic structure and plasma opacity calculations. The author demonstrated that the radial Schrödinger equation can be discretized into matrix form, converting the problem into an eigenvalue system solved using numerical linear algebra. The work implemented Hartree-Fock-Slater and LDA approaches on non-uniform radial grids, including relativistic corrections, to compute bound and continuum wavefunctions. These results were then used to evaluate dipole matrix elements and transition properties required for opacity modeling. Overall, the paper highlights matrix-based methods as a flexible and conceptually clear framework for realistic atomic and plasma computations.

Chapter 4: Methodology

4.1 Proposed Process

1- Load dataset using pandas.
2- Convert categorical values to numeric.
3- Convert DataFrame into NumPy matrix.
4- Analyze matrix shape.
5- Perform matrix operations.
6- Train simple ML model.

4.2 Dataset

The dataset used in this research is **xAPI-Edu-Data**, available on Kaggle.

The dataset contains student academic performance records collected using the Experience API (xAPI). It includes:

- Demographic features (gender, nationality)
- Academic background
- Behavioral attributes (raised hands, visited resources)
- Final class label (Low, Middle, High)

The dataset consists of multiple categorical and numerical features, making it suitable for demonstrating preprocessing and matrix transformation steps.

Chapter 5: Experimental Design and Settings

5.1 Implementation Details

The experiment was implemented in Python using:

- NumPy for matrix representation and linear algebra
- pandas for dataset manipulation
- scikit-learn for encoding and model training

Steps performed:

1. The dataset was loaded using pandas read_csv().
2. Categorical features were encoded using label encoding or one-hot encoding.
3. The processed DataFrame was converted to a NumPy array using .values or .to_numpy().
4. Matrix shape was examined using .shape.
5. Basic matrix operations were performed:
    - Matrix transpose
    - Mean vector computation
    - Covariance matrix computation
    - Matrix multiplication
6. A simple classification model was trained using scikit-learn.

Training/testing split was performed using 80% training and 20% testing data.

Chapter 6: Results and Discussion

6.1 Results

After preprocessing:

- The dataset was successfully transformed into a numerical matrix $X \in \mathbb{R}^{m \times n}$.
- Matrix shape confirmed correct representation of samples and features.
- Covariance matrix analysis showed variability among behavioral features.

- The ML model achieved reasonable classification accuracy equals 84% on predicting student performance levels.

The matrix operations verified:

- Transpose dimensions changed from (m × n) to (n × m).
- Matrix multiplication rules required inner dimensions to match.
- Feature scaling improved model stability.

6.2 Discussion

The experiment demonstrates that proper matrix representation is essential before any ML training process. Most ML errors arise from:

- Incorrect reshaping of vectors
- Confusion between row vectors and column vectors
- Mismatch between model input dimension and dataset features

The use of NumPy ensures computational efficiency, while pandas simplify preprocessing. The integration of these tools forms the standard AI workflow used in research and industry.

Chapter 7: Conclusion

This research examined the mathematical foundation of data representation in Artificial Intelligence using NumPy and matrix operations. Using the xAPI-Edu-Data dataset, we demonstrated how raw educational records are transformed into structured numerical matrices suitable for Machine Learning.

The study highlights that:

- Data must be numerically encoded.
- Matrix shape determines model compatibility.
- Linear algebra operations form the computational core of AI systems.

Understanding scalars, vectors, matrices, and tensors is not optional in AI — it is fundamental. Future work may extend this research to tensor-based deep learning models and high-dimensional feature analysis.

References

[1] E. Micha, "How Proportional Representation Can Shape Artificial Intelligence," *AI Magazine,* vol. 46, p. 10, Dec 2025.

[2] C. R. Harris, K. J. Millman, S. J. v. d. W. R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. v. Kerkwijk, M. Brett and A, Array Programming with NumPy, 2020.

[3] C. Bowen, "Matrix methods for opacity calculations," *High Energy Density Physics,* vol. 52, p. 101135, 2024.